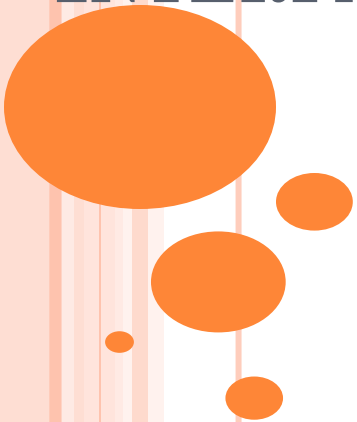
Interfaces gráficas con streamlit



Fundamentos

- **Librería para la construcción de interfaces gráficas en Python, basadas en Web.**

- **Se instala con:**

```
>pip install streamlit
```

- **Para utilizarla en un programa, debe ser importada:**

```
import streamlit as st
```

- **Para ejecutar un .py basado en streamlit:**

```
>streamlit run archivo.py
```

Antonio Martín Sierra

Escritura de datos

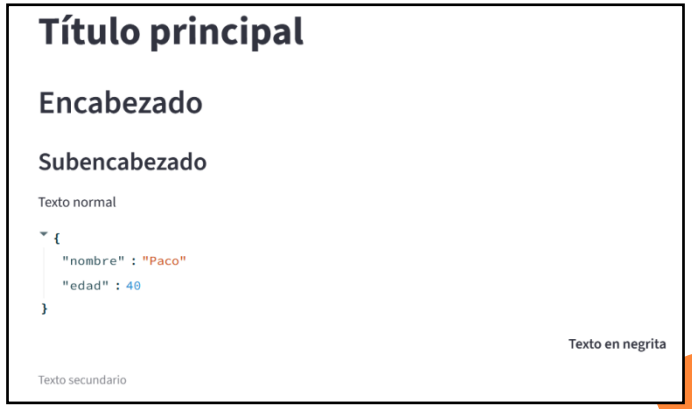
- **Streamlit proporciona distintos métodos para mostrar datos en la página:**
  - **text(texto). Muestra texto plano sin formato**
  - **write(objeto). Muestra un objeto Python respetando su formato**
  - **markdown(texto). Muestra texto con formato markdown**
  - **title(texto), header(texto) y subheader(texto). Muestran texto con diferentes tamaños y grosores**
  - **caption(texto). Texto semitransparente**

Antonio Martín Sierra

Ejemplo

```
import streamlit as st

st.title("Título principal")
st.header("Encabezado")
st.subheader("Subencabezado")
st.text("Texto normal")
st.write({"nombre":"Paco","edad":40})
#con un arterisco cursiva, con dos negrita
# y con tres negrita y cursiva
st.markdown("**Texto en negrita**",text_alignment="right")
st.caption("Texto secundario")
```



Antonio Martín Sierra

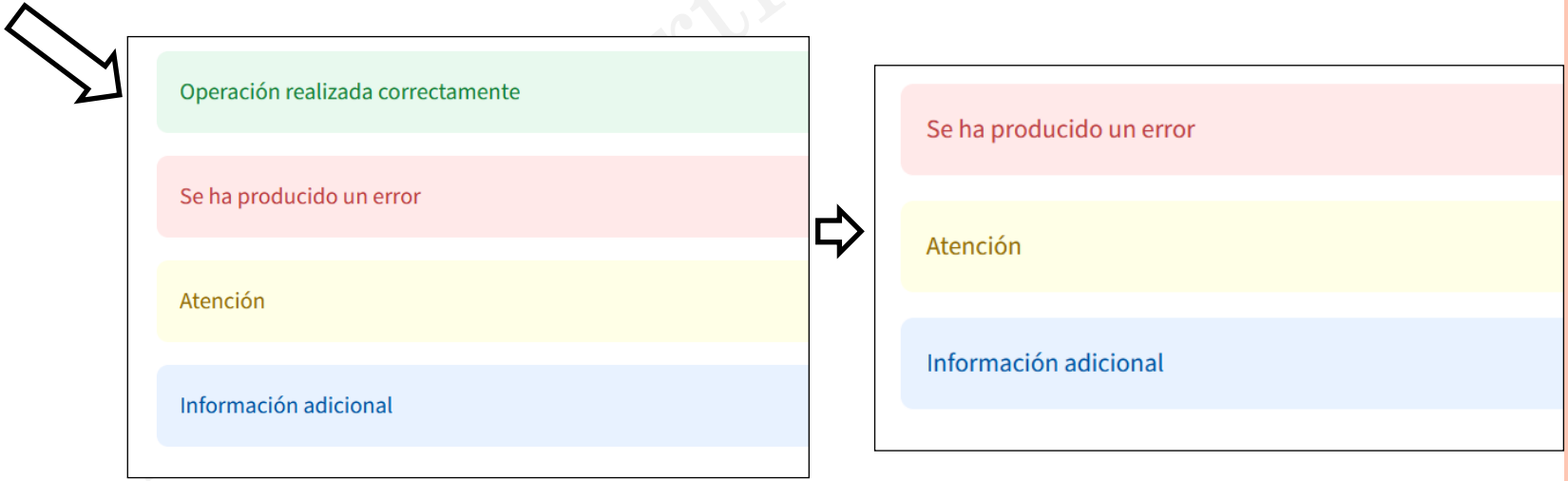
Mensajes

- Disponemos de varios métodos para mostrar mensajes de error, advertencia, etc.
- Se mostrarán en la página con distintos colores de fondo.
- Los métodos son: success(texto), error(texto), warning(texto) e info(texto)
- Los mensajes permanecen fijos, se puede utilizar time.sleep() para asignar un tipo de espera y hacerlo desaparecer con empty(mensaje)

Antonio Martín Sierra

Mensajes. Ejemplo

```
import streamlit as st
import time
mensaje=st.success("Operación realizada correctamente")
st.error("Se ha producido un error")
st.warning("Atención")
st.info("Información adicional")
time.sleep(3)
mensaje.empty() #desaparece a los tres segundos
```



Martín Sierra

Controles básicos

- **A traves de los siguientes métodos podemos generar los controles habituales que se emplean en una interfaz Web para interaccionar con el usuario:**
  - **text_input(texto). Campo de texto con un mensaje asociado. Devuelve el texto introducido**
  - **number_input(texto). Similar al anterior, pero solo admite valores numéricos**
  - **check_box(texto). Genera un checkbox. Devuelve un boolean indicando si está o no marcado**
  - **radio(texto, lista). Genera un grupo de radiobuttons a partir de la lista. Devuelve el seleccionado**
  - **selectbox(texto, lista). Genera una lista desplegable con el contenido de la lista**

Antonio Martín Sierra

Botón de pulsación

- **Se genera con el método button(texto).**
- **Devuelve True cuando se pulsa, por lo que debe utilizarse como condición de un if para definir la acción a realizar:**

```
if st.button("Pulsar"):
    st.success("Bienvenido a Streamlit")
```

- **El método form_submit_button(texto) genera un botón submit, empleado en formularios. Devuelve True cuando se ha pulsado**

Antonio Martín Sierra

Tablas

- **El método table() permite generar una tabla a partir del contenido de una lista:**

```
import streamlit as st
aula = [
    {"id": 1, "nombre": "Juan", "edad": 25},
    {"id": 2, "nombre": "Ana", "edad": 30},
    {"id": 3, "nombre": "Pedro", "edad": 22}
]
st.table(aula)
```



id	nombre	edad
1	Juan	25
2	Ana	30
3	Pedro	22

Antonio Martín Sierra

Pestañas

- **Permiten organizar la interfaz gráfica en varias vistas.**
- **Las pestañas se crean con el método tabs:**

```
tab1, tab2 = st.tabs(
    ["Alta", "Consulta"]
)
```

- **Para establecer el contenido de un tab, definimos un contexto para cada pestaña con with y en él definimos su contenido:**

```
with tab1:
    nombre=st.text_input("Introduce nombre: ")
    curso=st.text_input("Introduce curso: ")
    nota=st.number_input("Introduce nota: ",min_value=0,max_value=10)
    if st.button("Guardar"):
        alumnos.append({"nombre":nombre,"curso":curso,"nota":nota})
```

Adrián Martín Sierra

Contenedores y columnas

- **Las pestañas no son los únicos elementos que tiene streamlit para organizar la interfaz.**
- **Podemos utilizar contenedores para agrupar controles y columnas para dividir la interfaz:**

```
with st.container(border=True):
    st.title("Datos del alumno")
    st.text_input("Nombre")
    st.text_input("Curso")
col1, col2 = st.columns(2)
with col1:
    st.write("Contenido de la izquierda1")
    st.write("Contenido de la izquierda2")
    st.write("Contenido de la izquierda3")
with col2:
    st.write("Contenido de la derecha1")
    st.write("Contenido de la derecha2")
    st.write("Contenido de la derecha3")
```



dataframe y data_editor

- **El método *dataframe()* presenta los datos en forma de tabla, pero permite ordenar y obtener cálculos sobre las columnas.**
- **El método *data_editor()* permite modificar el contenido de las celdas de la tabla. Posteriormente, se puede volcar el nuevo contenido en el origen de datos:**

```
import streamlit as st
datos = [
    {"producto": "Portátil", "precio": 850},
    {"producto": "Monitor", "precio": 250},
    {"producto": "Teclado", "precio": 45}
]
resultado=st.data_editor(datos)
if st.button("Guardar"):
    datos=resultado  #actualización del origen de datos
    st.write(datos)
```